

一、使用的 AI 模型

主要使用 ChatGPT 作为辅助学习和问题排查工具。

二、使用的提示词

- 1.怎么配置部署 Ubuntu 环境
- 2.“SGLang 应该怎么安装和运行？”
- 3.“这个 CUDA 报错是什么原因？”
- 4.“如何检查 CUDA、nvcc 和相关 Python 包的版本？”
- 5.“Mooncake FAST’25 的 trace 数据应该怎么使用？”
- 6.“如何根据 input_length 和 output_length 构造测试请求？”
- 7.“这个作业的流程应该怎么设计？”
- 8.把实际的终端报错信息提供给 AI，让 AI 根据错误信息分析原因

三、AI 对本次学习的帮助

- 1.理解新的知识。刚开始接触 SGLang 时，我对 KV Cache 等概念并不熟悉。通过让 AI 用比较容易理解的方式解释，我能够先建立一个基本的认识，再结合实际运行过程进行理解。
- 2.排查环境和代码问题。面对比较复杂的终端报错时，我先把错误信息提供给 AI，然后根据它提供的排查思路检查版本、路径和环境变量，再自己执行命令验证。
- 3.完善实验代码和作业材料。比如在完成 Mooncake Trace 测试时，AI 帮助我理解 trace 文件中的 input_length 和 output_length，并协助设计请求生成和 Poisson 到达过程。实验过程中出现输入长度超过模型限制的问题后，我又根据实际运行结果修改程序，最终完成了测试。

四、学习哪方面用到 AI

1.SGLang 框架

通过 AI 了解 SGLang 的基本使用方法，以及在线推理服务的启动、请求发送和运行流程。

2.大语言模型推理原理

使用 AI 帮助理解 Tokenizer、Scheduler、Decode、Sampling、KV Cache、等概念，以及它们在一次模型推理中的作用。

3.环境配置和错误排查

在安装和运行 SGLang 的过程中遇到了 CUDA、FlashInfer、动态库以及版本不匹配等问题，通过 AI 分析报错信息，并结合终端实际结果一步步排查和修改。

4.实验代码和测试方法的学习

在 Mooncake FAST'25 trace 实验中，通过 AI 学习如何读取 trace 数据、根据 input_length 和 output_length 构造测试请求，以及如何使用 Poisson 到达过程模拟请求。

五、误导

环境配置涉及多个 CUDA 组件和动态库，不同环境下的安装路径也可能不同，Ai 刚开始给我的解决方法不能处理版本冲突的问题，是经过一遍遍的报错才发现问题。